Unit – 2 Advance PHP and File Management

2.1 File Handling and Directories

2.1.1 Including files using include and require

It is possible to insert the content of one PHP file into another PHP file (before the server executes it), with the include or require statement.

There are two functions that help us to include files:

include():

This function is used to copy all the contents of a file called within the function, text wise into a file from which it is called. This happens before the server executes the code.

Syntax

include '*filename*';

or

require '*filename*';

Example:

even.php

```
<?php

 // File to be included
 echo "Hello from PHP";
?>
```

Now, try to include this file into another PHP file index.php file, will see the contents of both the file are shown.

- **index.php**

```
<?php
   include("even.php");
   echo "<br>Above File is Included"
?>
```

output

Hello from PHP

Above File is Included

require():

The **require()** function in PHP is basically used to include the contents/code/data of one PHP file to another PHP file. During this process, if there are any kind of errors then this **require()** function will pop up a warning along with a fatal error and it will immediately stop the execution of the script. In order to use this **require()** function, we will first need to create two PHP files.

Example: This example is using the require() function in PHP.

- even.php

```
<?php
 // File to be included
 echo "Hello GeeksforGeeks";
?>
```

Now, if we try to include this file using require() function this file into a web page we need to use a index.php file. We will see that the contents of both files are shown.

include() vs require(): Both functions act as same and produce the same results, but if by any chance a fatal error arises, then the difference comes to the surface, which we will see following this example. Consider the following code:

index.php

```
<?php
   require("even.php");
   echo "<br>Above File is Required"
?>
```

```
<?php
   include("even.php");
   echo "<br>Above File is Included"
?>
```

Warning: include(even.php): failed to open stream: No such file or directory in **C:\xampp\htdocs\test\index.php** on line **2**

Warning: include(): Failed opening 'even.php' for inclusion (include_path='.;C:\xampp\php\PEAR') in **C:\xampp\htdocs\test\index.php** on line **2**

Above File is Included

include()	require()
The include() function does not stop the execution of the script even if any error occurs.	The require() function will stop the execution of the script when an error occurs.
The include() function does not give a fatal error.	The require() function gives a fatal error
The include() function is mostly used when the file is not required and the application	The require() function is mostly used when the file is mandatory for the

should continue to execute its process when the file is not found.	application.
The include() function will only produce a warning (E_WARNING) and the scrip t wil l contin ue execute. To	The require() produc e will A fatal Erro r (E_COMPILE_ERROR) along with the warning.

2.1.2 File operation

File handling is needed for any application. For some tasks to be done file needs to be processed. File handling in PHP is similar as file handling is done by using any programming language like C. PHP has many functions to work with normal files. Those functions are:

1)fopen()

The fopen() function in PHP is used to open a file or URL. It returns a file pointer that can be used with other functions like fread(), fwrite(), and fclose(). The function allows developers to interact with files in various ways, including reading, writing, and appending data.

Syntax:

```
resource fopen ( $file, $mode, $include_path, $context)
```

Parameters:

- ***$file*:** It is a mandatory parameter which specifies the file.
- ***$mode*:** It is a mandatory parameter which specifies the access type of the file or stream
  .
- It can have the following possible values:
  - **"r":** It represents Read only. It starts at the beginning of the file.
  - **"r+":** It represents Read/Write.It starts at the beginning of the file.
  - **"w":** It represents Write only.It opens and clears the contents of file or create a new file if it doesn't exist.
  - **"w+":** It represents Read/Write. It opens and clears the contents of file or creates a new file if it doesn't exist.
  - **"a":** It represents Write only. It opens and writes to the end of the file or creates a new file if it doesn't exist.
  - **"a+":** It represents Read/Write. It preserves the file's content by writing to the end of the file.

- **"x":** It represents Write only. It creates a new file and returns FALSE and an error if the file already exists.
- **"x+":** It represents Read/Write.It creates a new file and returns FALSE and an error if file already exists.

- ***$include_path*:** It is an optional parameter which is set to 1 if you want to search for the file in the include_path (Ex. php.ini).

- ***$context*:** It is an optional parameter which is used to set the behavior of the stream.

2) fread():
The fread() function is used to read a file or a stream of data in PHP. It reads up to a specified number of bytes from an open file. The function returns the content that has been read, which can then be processed or displayed.

Syntax:
string fread ($file, $length)

Code:	**Output:**

<?php
$file = fopen("myfile.txt", "r");
 	
if ($file) {
  $content = fread($file, filesize("myfile.txt"));
  fclose($file);
  echo $content;
} else {
  echo "Failed to open the file.";
}
?>
	This is simple text

3) fwrite( )
The fwrite() function in PHP is used to write data (string or binary) to an open file. Before using fwrite(), you need to open the file with fopen(). Once the file is open, fwrite() sends your data to that file.

Writing to a File
Here's an example of opening a file in write-only mode ('w') and writing some data to it:

<?php
$file = fopen("example.txt", "w");
if ($file) {
  fwrite($file, "This is a sample text.");
  fclose($file);
  echo "Data written to file successfully.";
} else {
  echo "Failed to open file for writing.";
}
?>
	Output:
	This is a Sample text.

Or <?php $file = fopen("myfile.txt","w"); echo fwrite($file,"Hello World. Testing!");	Output: 21

fclose($file); ?>	

Appending Data to a File

If you want to append data to an existing file, you can open it in append mode ('a'):

Code:	Output:

<?php
$file = fopen("example.txt", "a");
if ($file) {
/ fwrite($file, "\nAppended text.");
fclose($file);
echo "Data appended to file successfully.";
} else {
echo "Failed to open file for appending.";
}
?>
	This is a Simple text.
	ASSC

4)fclose()function

The fclose() function in PHP closes a file that was previously opened by fopen(). Closing a file releases the resource associated with it and makes sure all the data written to the file is properly saved.

Syntax:

bool fclose(resource $handle)

- $handle: The file resource you want to close. This is the value returned by fopen().

2.1.3 File upload using using $_FILES and move_uploaded_file

PHP allow you to upload any type of a file i.e. image, binary or text files.etc..,PHP has one in built global variable i.e. **$_FILES**, it contains all information about file.By the help of **$_FILES** global variable, we can get file name, file type, file size and temparary file name associated with file.

- **$_FILES['filename']['name']** - returns file name.
- **$_FILES['filename']['type']** - returns MIME type of the file.
- **$_FILES['filename']['size']** - returns size of the file (in bytes).
- **$_FILES['filename']['tmp_name']** - returns temporary file name of the file which was stored on the server.

First Configure the php.ini File by ensure that PHP is configured to allow file uploads. In your php.ini file, search for the file_uploads directive, and set it to On i.e. file_uploads = On

- **Step 1: Creating an HTML form to upload the file**

  Below is the HTML source code for the HTML form for uploading the file to the server. In the HTML <form> tag, we are using "enctype='multipart/form-data" which is an encoding type that allows files to be sent through a POST method. Without this encoding, the files cannot be sent through the POST method. We must use this enctype if you want to allow users to upload a file through a form.

File_uploading.php

```
<!DOCTYPE html>
<html>
  <head>
    <title>File Uploading</title>
  </head>
  <body>
    <h2>File Upload Form</h2>
<form method = "POST" action = "fileupload.php" enctype="multipart/form-data">
  <label for="file">File name:</label>
  <input type="file" name="uploadfile" />
  <input type="submit" name="submit" value="Upload" />
</form>
  </body>
</html>
```

Output:

Step 2:Creating an Upload Script

The **fileupload.php** script receives the uploaded file. The file data is collected in a superglobal variable **$_FILES**. Fetch the name, file type, size and the tmp_name attributes of the uploaded file.

Fileupload.php

```
<?php
  echo "<b>File to be uploaded: </b>" . $_FILES["uploadfile"]["name"] . "<br>";
  echo "<b>Type: </b>" . $_FILES["uploadfile"]["type"] . "<br>";
  echo "<b>File Size: </b>" . $_FILES["uploadfile"]["size"]/1024 . "<br>";
```

```
    echo "<b>Store in: </b>" . $_FILES["uploadfile"]["tmp_name"] . "<br>";

    if (file_exists($_FILES["uploadfile"]["name"]))
  {
     echo "<h3>The file already exists</h3>";
    }
  Else
  {
  move_uploaded_file($_FILES["uploadfile"]["tmp_name"],
  $_FILES["uploadfile"]["name"]);
     echo "<h3>File Successfully Uploaded</h3>";
    }
  ?>
```

Output:

Click the File button, browse to the desired file to be uploaded, and click the Upload button.

The server responds with the following message –

File to be uploaded: welcome.png
Type: image/png
File Size: 35.7529296875
Store in: C:\xampp\tmp\phpCC37.tmp

File Successfully Uploaded

2.1.4 file download using php header

Why Use PHP for File Downloads?

PHP makes it easy to manage file downloads. It allows you to manage file access and add security measures. PHP can be used for –

- Allow users to download files such as PDFs, images, and videos.
- Secure files so that only logged-in users can access them.
- Monitor the number of times a file has been downloaded.

Example

The following PHP script shows the usage of readfile() function.

The **readfile() function** returns the number of bytes read or false even it is successfully completed or not.

To download a file

the Content-Type response header should be set to application/octect-stream.
This MIME type is the default for binary files.

Browsers usually don't execute it, or even ask if it should be executed.

Additionally, setting the Content-Disposition header to attachment prompts the "Save As" dialog to pop up.

```
<?php

  $filePath = 'myfile.txt';

  // Set the Content-Type header to application/octet-stream
  header('Content-Type: application/octet-stream');

  // Set the Content-Disposition header to the filename of the downloaded file
        header('Content-Disposition: attachment; filename="'.
basename($filePath).'"');

  // Read the contents of the file and output it to the browser.
  readfile($filePath);
?>
```

2.1.5 Directory Handling in PHP

PHP includes a set of directory functions to deal with the operations, like, listing irectory contents, and create/ open/ delete specified directory and etc. These basic functions are listed below.

- *mkdir()*: To make new directory.
- *opendir()*: To open directory.
- *readdir()*: To read from a directory after opening it.
- closedir(): To close directory with resource-id returned while opening.

Creating Directory

Parameters

Here are the required and optional parameters of the **mkdir()** function –

Sr.No	Parameter & Description
1	**$pathname(Required)** It is the path where the new directory will be created.
2	**$mode(Optional)** It is the permission for the new directory. Default is 0777.
3	**$recursive(Optional)** If set to true, it will create nested directories and default is false.
4	**$context(Optional)** It is a context stream resource.

Example:

```
<?php
  // Specify the path inside the mkdir function
  mkdir("/xampp/htdocs/testing1");

  // Print the message
  echo "Directory created successfully!!!";
?>
```

Directory created successfully!!!

Listing the Directory Content

For listing the contents of a directory, we require two of the above-listed PHP directory functions, these are, *opendir()* and *readdir()*. There are two steps in directory listing using the PHP program.

- Step 1: Opening the directory.
- Step 2: Reading content to be listed one by one using a PHP loop.

Step 1: Opening Directory Link

opendir() function in PHP is an inbuilt function which is used to open a directory handle. The path of the directory to be opened is sent as a parameter to the opendir() function and it returns a directory handle resource on success, or FALSE on failure. The syntax will be,

```
<?php
opendir($directory_path,  $context);
?>
```

Step 2: Reading Directory Content

The readdir() function in PHP is an inbuilt function which is used to return the name of the next entry in a directory. The method returns the filenames in the order as they are stored in the filename system. The directory handle is sent as a parameter to the readdir() function and it returns the entry name/filename on success or False on failure.

Syntax:

readdir(dir_handle)

Parameters Used: The readdir() function in PHP accepts one parameter.

- **dir_handle** : It is a mandatory parameter which specifies the handle resource previously opened by the opendir() function.

```
<?php

// opening a directory
$dir_handle = opendir("/xampp/htdocs/");

// reading the contents of the directory
while(($file_name = readdir($dir_handle))
!== false)
{
echo("File Name: " . $file_name);
echo "<br>" ;
}

// closing the directory
closedir($dir_handle);
?>
```

```
File Name: gfg.jpg
File Name: ..
File Name: article.pdf
File Name: .
File Name: article.txt
```

And, thereby executing the above code sample, we can list the content of a directory as expected.

Step 3: Closing Directory

The PHP Directory **closedir()** function stands for "Close Directory". This function can be used to close a directory handle.

Syntax:

```
<?php
$directory_handle = opendir($directory_path);
...
...
closedir($directory_handle);
?>
```

Example

Open a directory, read its contents, then close:

Code:

```
<?php

// Opening a directory
$dir_handle = opendir("/xampp/htdocs/");
if(is_resource($dir_handle))
{
        echo("Directory Opened Successfully.");
}
// closing the directory
closedir($dir_handle);
?>
```

```
filename:.
filename:..
filename:applications.html
filename:bitnami.css
filename:codeigniter
filename:codeigniter4-framework-v4.6.1-0-gd021b04.zip
filename:dashboard
filename:favicon.ico
filename:fileupload
filename:img
filename:index.php
filename:myproj1
filename:myproject
filename:php_practicle_list
filename:webalizer
filename:wordpress
filename:xampp
```

rmdir():

The PHP Filesystem rmdir() function is used to remove an empty directory, and return true on success, or false on failure.

Code:

```
<?php

  $dir = "/PhpProject/Myfolder";



  if (is_dir($dir)) {

    if (rmdir($dir)) {

      echo "Directory 'Myfolder' removed successfully.";
```

Directory 'Myfolder' removed successfully.

```
    } else {

       echo "Error removing directory
'Myfolder'.";

    }

  } else {

    echo "Directory 'Myfolder' does
not exist.";

  }

?>
```

2.2 Forms, Filters and JSON

2.2.1 Form Designing and handling forms

Form handling is the process of collecting and processing information that users submit through HTML forms. In PHP, we use special tools called $_POST and $_GET to gather the data from the form. Which tool to use depends on how the form sends the data either through the POST method (more secure, hidden in the background) or the GET method (data is visible in the URL).

- **Collecting Data:** Retrieving form data using PHP.
- **Validating Data:** Ensuring that the input meets expected formats.
- **Sanitizing Data:** Cleaning up the data to prevent malicious content.
- **Processing Data**: Using the data for its intended purpose (e.g., saving to a database, sending an email, etc.).
- **Returning a Response**: Displaying feedback to the user or redirecting them to another page

Attributes of Form Tag:

Attribute	Description
name or id	It specifies the name of the form and is used to identify individual forms.
Action	It specifies the location to which the form data has to be sent when the form is submitted.
Method	It specifies the HTTP method that is to be used when the form is submitted. The possible values are **get** and **post**. If **get** method is used, the form data are visible to the users in the url. Default HTTP method is **get**.
encType	It specifies the encryption type for the form data when the form is submitted.

Novalidate	It implies the server not to verify the form data when the form is submitted.

Controls used in forms: Form processing contains a set of controls through which the client and server can communicate and share information. The controls used in forms are:

- **Textbox:** Textbox allows the user to provide single-line input, which can be used for getting values such as names, search menu and etc.
- **Textarea:** Textarea allows the user to provide multi-line input, which can be used for getting values such as an address, message etc.
- **DropDown:** Dropdown or combobox allows the user to provide select a value from a list of values.
- **Radio Buttons:** Radio buttons allow the user to select only one option from the given set of options.
- **CheckBox:** Checkbox allows the user to select multiple options from the set of given options.
- **Buttons:** Buttons are the clickable controls that can be used to submit the form.

All the form controls given above is designed by using the **input** tag based on the **type** attribute of the tag. In the below script, when the form is submitted, no event handling mechanism is done. Event handling refers to the process done while the form is submitted. These event handling mechanisms can be done by using javaScript or

PHP. However, JavaScript provides only client-side validation. Hence, we can use PHP for form processing.

PHP **htmlspecialchars**() Function
Example
Convert the predefined characters "<" (less than) and ">" (greater than) to HTML entities:

Convert the predefined characters "<" (less than) and ">" (greater than) to HTML entities:

```php
<?php
$str = "This is some <b>bold</b> text.";
echo htmlspecialchars($str);
?>
```

The HTML output of the code above will be (View Source):

```
<!DOCTYPE html>
<html>
<body>
This is some &lt;b&gt;bold&lt;/b&gt; text.
</body>
</html>
```

The browser output of the code above will be:

```
This is some <b>bold</b> text.
```

```
<html>
<body>
  <form method="post" action="<?php
echo htmlspecialchars($_SERVER["PHP_SELF"]); ?>">
    <table>
      <tr>
        <td>Full Name:</td>
        <td><input type="text" name="fname"></td>
      </tr>
      <tr>
        <td>Email Address:</td>
        <td><input type="email" name="femail"></td>
      </tr>
      <tr>
        <td>Age:</td>
        <td><input type="text" name="fage"></td>
      </tr>
      <tr>
        <td>Feedback:</td>
        <td><textarea name="feedback" rows="4" cols="40"></textarea></td>
```

```
        </tr>
        <tr>
          <td>Gender:</td>
          <td>
            <input type="radio" name="user_gender" value="female">Female
            <input type="radio" name="user_gender" value="male">Male
          </td>
        </tr>
        <tr>
          <td colspan="2"><input type="submit" name="submit" value="Submit"></td>
        </tr>
      </table>
    </form>


    <?php
      $fullname = $user_email = $user_gender = $user_feedback = $user_age = "";

      // Check if the form was submitted
      if ($_SERVER["REQUEST_METHOD"] == "POST") {
        // Only process the POST data if the form is submitted
        if (isset($_POST['fname'])) {
          $fullname = htmlspecialchars($_POST['fname']);
        }
        if (isset($_POST['femail'])) {
          $user_email = htmlspecialchars($_POST['femail']);
        }
        if (isset($_POST['fage'])) {  // Corrected variable to user_age
          $user_age = htmlspecialchars($_POST['fage']);
        }
        if (isset($_POST['feedback'])) {
          $user_feedback = htmlspecialchars($_POST['feedback']);
        }
        if (isset($_POST['user_gender'])) {
          $user_gender = htmlspecialchars($_POST['user_gender']);
        }
      }

      // Display the submitted details
      echo "<h2>Details Submitted:</h2>";
      echo "Full Name: " . $fullname . "<br>";
      echo "Email: " . $user_email . "<br>";
      echo "Age: " . $user_age . "<br>";
      echo "Feedback: " . $user_feedback . "<br>";
      echo "Gender: " . $user_gender;
    ?>
  </body>
</html>
```

Full Name:

Email Address:

Age:

Feedback:

Gender: Female Male

Submit

Details Submitted:

Full Name: GEET
Email: GEET@GMAIL.COM
Age: 30
Feedback: DF
Gender: female

2.2.2. Server Side form Validation in PHP

Server-side validation is another way to validate an HTML Form. In Server Side validation we can validate empty filed, input length, numeric value, valid email id and many more.

```
<?php
$nameErr = $emailErr = "";


$name = $email = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
 if (empty($_POST["name"])) {
  $nameErr = "Name is required";
 } else {
  $name = test_input($_POST["name"]);
  // check if name only contains letters and whitespace
  if (!preg_match("/^[a-zA-Z ]*$/",$name)) {
   $nameErr = "Only letters and white space allowed";
  }
 }

 if (empty($_POST["email"])) {
  $emailErr = "Email is required";
```

```
  } else {
    $email = test_input($_POST["email"]);
    // check if e-mail address is well-formed
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
      $emailErr = "Invalid email format";
    }
  }
   // If no errors, process the data (e.g., insert into database)
  if (empty($nameErr) && empty($emailErr)) {
     // Process the data (e.g., insert into database)
     echo "Form submitted successfully!";
     // You would typically handle the data insertion here
  }
}

function test_input($data) {
  $data = trim($data);
  $data = stripslashes($data);
  $data = htmlspecialchars($data);
  return $data;
}
?>

<p><span class="error">* required field</span></p>
<form method="post" action="<?php echo
htmlspecialchars($_SERVER["PHP_SELF"]);?>">
  Name: <input type="text" name="name">
  <span class="error">* <?php echo $nameErr;?></span>
  <br><br>
  E-mail: <input type="text" name="email">
  <span class="error">* <?php echo $emailErr;?></span>
  <br><br>
  <input type="submit" name="submit" value="Submit">
</form>
```

Explanation:

1. $_SERVER["REQUEST_METHOD"] == "POST": Checks if the form was submitted using the POST method.
2. empty($_POST["name"]): Checks if the "name" field is empty.
3. test_input($_POST["name"]): Calls a function to sanitize the input. This function:
    - trim() removes whitespace from the beginning and end of the string.
    - stripslashes() removes backslashes.
    - htmlspecialchars() converts special characters to HTML entities, preventing XSS attacks.
4. preg_match("/^[a-zA-Z]*$/",$name): Validates that the name contains only letters and whitespace using a regular expression.
5. filter_var($email, FILTER_VALIDATE_EMAIL): Uses PHP's built-in filter to validate the

email format.

6. Error Handling: If any validation fails, an error message is stored in the corresponding error variable (e.g., $nameErr).
7. Displaying Errors: The error messages are displayed next to the respective form fields.
8. Data Processing: If all validations pass (i.e., $nameErr and $emailErr are empty), the code proceeds to process the data (e.g., insert into the database).
9. htmlspecialchars($_SERVER["PHP_SELF"]): Ensures the form is submitted to the same page, which allows displaying error messages on the same page.

PHP Filters

A PHP Filter is used to validate and filter data coming from insecure sources.

To test, validate, and filter user input or custom data is an important part of any web application.

The PHP filter extension is designed to make data filtering easier and quicker.

Functions and Filters

To filter available, use one of the following filter function:

1.filter_var()- Filter a single variable with a specified filter.

Example, we validate an integer using the filter_var() function

Code:	**Integer is valid**
<?php **$int=1234;** **if(!filter_var($int,FILTER_VALIDATE_INT))** **{** **echo "Integer is not valid";** **} else {** **echo "Integer is valid";** **}** **?>**	

2.2.4 Parsing and generating JSON with json_encode() and json_decode()

Encodig JSON with json_encode()

The json_encode() function takes a PHP value as input and returns its JSON string representation. By default, it encodes PHP arrays as JSON objects if the array keys are strings, and as JSON arrays if the keys are sequential integers.

Code:

```
<?php
$data = array("name" => "John Doe", "age" => 30, "city" => "New York");
$json = json_encode($data);
echo $json; // Output: {"name":"John Doe","age":30,"city":"New York"}
?>
```

You can also use json_encode() to encode other data types like integers, floats, booleans, and strings directly.

Code:

```
<?php
$number = 123;
$json_number = json_encode($number);
echo $json_number; // Output: 123
?>
```

Decoding JSON with json_decode()

The json_decode() function takes a JSON string as input and returns its PHP representation. By default, it returns a PHP object. If you want an associative array instead, you can pass true as the second argument.

Code:

```
<?php
$json = '{"name":"John Doe","age":30,"city":"New York"}';
$object = json_decode($json);
echo $object->name; // Output: John Doe

$array = json_decode($json, true);
echo $array["name"]; // Output: John Doe
?>
```

2.3 Cookies and Sessions, Email

Session

When you work with an application, you open it, do some changes, and then you close it. This is much like a Session. The computer knows who you are. It knows when you start the application and when you end. But on the internet there is one problem: the web server does not know who you are or what you do, because the HTTP address doesn't maintain state.

Session variables solve this problem by storing user information to be used across multiple pages (e.g. username, favorite color, etc). By default, session variables last until the user closes the browser.

So; Session variables hold information about one single user, and are available to all pages in one application.

How are sessions better than cookies?

Although cookies are also used for storing user related data, they have serious security issues because cookies are stored on the user's computer and thus they are open to attackers to easily modify the content of the cookie. Addition of harmful data by the attackers in the cookie may result in the breakdown of the application. Apart from that cookies affect the performance of a site since cookies send the user data each time the user views a page. Every time the browser requests a URL to the server, all the cookie data for that website is automatically sent to the server within the request.

Below are different steps involved in PHP sessions:

Start a PHP Session

A session is started with the session_start() function.

Session variables are set with the PHP global variable: $_SESSION.

Below is the PHP code to start a new session:

```
<?php
session_start();
?>
```

demo_session1.php

<?php **// Start the session** **session_start();** **?>** **<!DOCTYPE html>** **<html>** **<body>** **<?php** **// Set session variables** **$_SESSION["color"] = "pink";** **$_SESSION["animal"] = "cow";** **echo "Session variables are set.";** **?>** **</body>**	**Output** **Session variables are set.**

</html>	

Get PHP Session Variable Values

Code:	**Output**
<?php **session_start();** **?>** **<!DOCTYPE html>** **<html>** **<body>** **<?php** **// Echo session variables that were set on previous page** **echo "Favorite color is " . $_SESSION["color"] . ". ";** **echo "Favorite animal is " . $_SESSION["animal"] . ".";** **?>** **</body>** **</html>**	**Favorite color is pink.** **Favorite animal is cow.**

Destroy a PHP Session

To remove all global session variables and destroy the session, use session_unset() and session_destroy():

Code:

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// remove all session variables
session_unset();
```

```
// destroy the session
session_destroy();
?>

</body>
</html>
```

Cookies

A **cookie** in PHP is a small file with a maximum size of 4KB that the web server stores on the client computer. They are typically used to keep track of information such as a username that the site can retrieve to personalize the page when the user visits the website next time. A cookie can only be read from the domain that it has been issued from. Cookies are usually set in an HTTP header but JavaScript can also set a cookie directly on a browser.

Setting Cookie In PHP: To set a cookie in PHP, the **setcookie()** function is used. The setcookie() function needs to be called prior to any output generated by the script otherwise the cookie will not be set.

Syntax:

```
setcookie(name, value, expire, path, domain, security);
```

Parameters:

- **Name:** It is used to set the name of the cookie.
- **Value:** Value stored in the cookie

- **Expire:** Expiration time in Unix timestamp (e.g., time() + 3600 = 1 hour)
- **Path:** Path on the server where the cookie is available (default is /)
- **Domain:** It is used to specify the domain for which the cookie is available.
- **Security:** It is used to indicate that the cookie should be sent only if a secure HTTPS connection exists.

Below are some operations that can be performed on Cookies in PHP:

1.Creating Cookies: Creating a cookie named Auction_Item and assigning the value Luxury Car to it. The cookie will expire after 2 days(2 days * 24 hours * 60 mins * 60 seconds)

Example: This example describes the creation of the cookie in PHP.

Code:

```
<?php
setcookie("username", "assc123", time() + 3600); // expires in 1 hour
echo "Cookie is set!";
?>

<a>
<body>
<?php
echo "cookie is created." ?>
</body>
</html>
```

Output :cookie is created

2.Get a Cookie: It is always advisable to check whether a cookie is set or not before accessing its value. **Accessing Cookie Values**: For accessing a cookie value, the PHP $_COOKIE superglobal variable is used. It is an associative array that contains a record of all the cookies values sent by the browser in the current request.

Example: This example describes checking whether the cookie is set or not.

```
<?php
if (isset($_COOKIE['username'])) {
```

```
    echo "Username from cookie: " . $_COOKIE['username'];
} else {
    echo "No cookie found!";
}
?>
```

Output:

Username from cookie assc123

3.Deleting Cookies: The To delete a cookie in PHP, you need to use the setcookie() function and set the expiration time to a past time. This will cause the browser to remove the cookie.

```
<?php
// Set expiration time in the past to delete
setcookie("username", "", time() - 3600);
echo "Cookie deleted.";
?>
```

Output:

Cookie is deleted.

form.php — Username and password

Code:

```
<!DOCTYPE html>
<html>
<head>
 <title>Login Form</title>
</head>
<body>

 <h2>Login Form (using Cookies)</h2>
 <form action="login.php" method="post">
  <label>Username:</label>
  <input type="text" name="username" required><br><br>

  <label>Password:</label>
  <input type="password" name="password" required><br><br>

  <input type="submit" value="Login">
```

```
 </form>

</body>
</html>
```

Login.php

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
   // Get values from form
   $username = $_POST['username'];
   $password = $_POST['password'];

   // Set cookies (valid for 1 hour)
   setcookie("username", $username, time() + 3600, "/");
   setcookie("password", $password, time() + 3600, "/");

   echo " Cookies have been set successfully.<br><br>";

   echo "Stored in cookie:<br>";
   echo "Username: " . htmlspecialchars($username) . "<br>";
   echo "Password: " . htmlspecialchars($password) . "<br><br>";

   //delete cookie
  // setcookie("username", $username, time() - 3600, "/");
  // setcookie("password", $password, time() - 3600, "/");

   echo '<a href="welcome.php">Go to Welcome Page</a>';
} else {
   echo " Invalid request.";
}
?>
```

Welcome.php

```
<?php
if (isset($_COOKIE["username"])) {
   echo "Welcome back, " . $_COOKIE["username"] . "!";
} else {
   echo "Please login first.";
}
?>
```

Send Mail in PHP using PHPMailer function

PHPMailer is a code library and used to send emails safely and easily via PHP code from a web server. Sending emails directly via PHP code requires a high-level familiarity to SMTP standard protocol and related issues and vulnerabilities about Email injection for spamming. PHPMailer simplifies the process of sending emails and it is very easy to use.

- **isSMTP():** Set mailer to use SMTP.
- **isMail():** Set mailer to use PHP's mail function.
- **Host:** Specifies the servers.
- **SMTPAuth:** Enable/Disable SMTP Authentication.
- **Username:** Specify the username.
- **Password:** Specify the password.
- **SMTPSecure:** Specify encryption technique. Accepted values 'tls' or 'ssl'.
- **Port:** Specify the TCP port which is to be connected.

```
$mail->SMTPDebug = 2;                // Enable verbose debug output
$mail->isSMTP();                     // Set mailer to use SMTP
$mail->Host      = 'smtp.gfg.com;';  // Specify main SMTP server
$mail->SMTPAuth  = true;             // Enable SMTP authentication
$mail->Username  = 'user@gfg.com';   // SMTP username
$mail->Password  = 'password';       // SMTP password
$mail->SMTPSecure = 'tls';           // Enable TLS encryption, 'ssl' also accepted
$mail->Port      = 587;              // TCP port to connect to
```

Add the recipients of the mail.

```
$mail->setFrom('from@gfg.com', 'Name');          // Set sender of the mail
$mail->addAddress('receiver1@gfg.net');          // Add a recipient
$mail->addAddress('receiver2@gfg.com', 'Name');   // Name is optional
```

Add attachments (if any).

```
$mail->addAttachment('url', 'filename');    // Name is optional
```

Add the content.

- **isHTML():** If passed true, sets the email format to HTML.
- **Subject:** Set the subject of the Mail.
- **Body:** Set the contents of the Mail.
- **AltBody:** Alternate body in case the e-mail client doesn't support HTML.

Finally, send the email.

```
$mail->send();
```

Example

```
<?php
use PHPMailer\PHPMailer\PHPMailer;
use PHPMailer\PHPMailer\Exception;

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
   $name    = $_POST['name'];
   $email   = $_POST['email'];
    $message = $_POST['message'];

//Load Composer's autoloader (created by composer, not included with PHPMailer)
require 'PHPMailer/src/Exception.php';
require 'PHPMailer/src/PHPMailer.php';
require 'PHPMailer/src/SMTP.php';

//Create an instance; passing `true` enables exceptions
```

```
   $mail = new PHPMailer(true);


try {
   //Server settings
   $mail->isSMTP();                                  //Send using SMTP
   $mail->Host       = 'smtp.gmail.com';             //Set the SMTP server to send through
   $mail->SMTPAuth   = true;                         //Enable SMTP authentication
   $mail->Username   = 'jas.mca24@gmail.com';        //SMTP username
   $mail->Password   = 'secret';                     //SMTP password
   $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;         //Enable implicit
TLS encryption
   $mail->Port       = 587;                          //TCP port to connect to; use 587 if you have set
`SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS`


   //Recipients
   $mail->setFrom('jas.mca24@gmail.com', 'contact form');
   $mail->addAddress('jas.mca24@gmail.com', 'our website');    //Add a recipient



   //Content
$mail->isHTML(true);
      $mail->Subject = 'New Contact Form Submission';
      $mail->Body    = "
         <h3>Message from $name</h3>
         <p><strong>Email:</strong> $email</p>
         <p><strong>Message:</strong><br>$message</p>
      ";


      $mail->send();
      echo 'Thank you! Your message has been sent.';
   } catch (Exception $e) {
      echo "Message could not be sent. Error: {$mail->ErrorInfo}";
   }
}
else
{
   header('location:index.php');
```

```
   exit(0);
}

?>
```

Send Email Using Mail () function

PHP is a server side scripting language that is enriched with various utilities required. Mailing is one of the server side utilities that is required in most of the web servers today. Mailing is used for advertisement, account recovery, subscription etc. In order to send mails in PHP, one can use the mail() method.

Syntax:

```
bool mail(to , subject , message , additional_headers , additional_parameters)
```

Parameters: The function has two required parameters and one optional parameter as described below:

- **to**: Specifies the email id of the recipient(s). Multiple email ids can be passed using commas
- **subject**: Specifies the subject of the mail.
- **message**: Specifies the message to be sent.
- **additional-headers**(Optional): This is an optional parameter that can create multiple header elements such as From (Specifies the sender), CC (Specifies the CC/Carbon Copy recipients), BCC (Specifies the BCC/Blind Carbon Copy Recipients. **Note:** In order to add multiple header parameters one must use '\r\n'.
- **additional-parameters**(Optional): This is another optional parameter and can be passed as an extension to the additional headers. This can specify a set of flags that are used as the sendmail_path configuration settings.

SMTP.txt

php.ini file:

```
SMTP=smtp.gmail.com
smtp_port=587
sendmail_from = yourgmailaddress@gmail.com
sendmail_path ="\" c:\xampp\sendmail\sendmail.exe\" -t"
```

sendmail.ini:

```
[sendmail]
smtp_server=smtp.gmail.com
```

smtp_port=587

error_logfile=error.log

debug_logfile=debug.log

auth_username=yourgmailaddress@gmail.com

auth_password=yourapppassword

force_sender=yourgmailaddress@gmail.com

how to search google app auth_password

manage your google account->security->app password->outlook

Examples:

```
$to = "recipient@example.com";
$sub = "Generic Mail";
$msg = "Hello Geek! This is a generic email.";
$headers = 'From: sender@example.com' . "\r\n"
.'CC: another@example.com';
if(mail($to,$sub,$msg,$headers))
   echo "Your Mail is sent successfully.";
else
   echo "Your Mail is not sent. Try Again.";
```

Output :

Your Mail is sent successfully.

2.4 OOP and Exception Handling in PHP

object-oriented programming is a programming model organized around **Object rather than the actions** and **data rather than logic**. Basic terminology of OOP to revise are as follows:

- **Class –** This is a programmer-defined data type, which includes local functions as well as local data. You can think of a class as a template for making many instances of the same kind (or class) of object.
- **Object –** An individual instance of the data structure defined by a class. You define a class once and then make many objects that belong to it. Objects are also known as instance.
- **Member Variable –** These are the variables defined inside a class. This data will be invisible to the outside of the class and can be accessed via member functions. These variables are called attribute of the object once an object is created.
- **Member function –** These are the function defined inside a class and are used to access object data.
- **Inheritance –** When a class is defined by inheriting existing function of a parent class then it is called inheritance. Here child class will inherit all or few member functions and variables of a parent class.
- **Parent class –** A class that is inherited from by another class. This is also called a base class or super class.
- **Child Class –** A class that inherits from another class. This is also called a subclass

or derived class.

- **Polymorphism –** This is an object oriented concept where same function can be used for different purposes. For example function name will remain same but it take different number of arguments and can do different task.

- **Overloading –** a type of polymorphism in which some or all of operators have different implementations depending on the types of their arguments. Similarly functions can also be overloaded with different implementation.
- **Data Abstraction –** Any representation of data in which the implementation details are hidden (abstracted).
- **Encapsulation –** refers to a concept where we encapsulate all the data and member functions together to form an object.
- **Constructor –** refers to a special type of function which will be called automatically whenever there is an object formation from a class.
- **Destructor –** refers to a special type of function which will be called automatically whenever an object is deleted or goes out of scope.

Define a Class

A class is defined by using the class keyword, followed by the name of the class and a pair of curly braces ({}). All its properties and methods go inside the braces:

Syntax

```
<?php
class Fruit {
  // code goes here...
}
?>
```

Below we declare a class named Fruit consisting of two properties ($name and $color) and two methods set_name() and get_name() for setting and getting the $name property:

Define Objects

Objects of a class are created using the new keyword.

In the example below, $apple and $banana are instances of the class Fruit:
Example:

```
<?php
class Fruit {
  // Properties
  public $name;
  public $color;

  // Methods
  function set_name($name) {
    $this->name = $name;
  }
  function get_name() {
    return $this->name;
  }
}

$apple = new Fruit();
```

```
$banana = new Fruit();
$apple->set_name('Apple');
$banana->set_name('Banana');

echo $apple->get_name();
echo "<br>";
echo $banana->get_name();
?>
```

The $this keyword refers to the current object, and is only available inside methods.

2.4.2 Using constructors and property visibility

PHP - Constructors

A constructor is a special function or method in a class that is automatically called when an object of the class is created. The constructor is mainly used for initializing the object, i.e., setting the initial state of the object by assigning values to properties.

Notice that the construct function starts with two underscores (__)!

__construct() Method: __construct is a public magic method that is used to create and initialize a class object. __construct assigns some property values while creating the object. This method is automatically called when an object is created.

Properties:

- __construct is a public magic method.
- __construct is a method that must have public visibility
- __construct method can accept one and more arguments.
- __construct method is used to create an object.
- __construct method can call the class method or functions
- __construct method can call constructors of other classes also.

Syntax

```
<?php
class ClassName {
   public function __construct() {
      // Constructor code here
   }
}
?>
```

Default Parameter

By default, __construct() method has no parameters. The values passed to the default constructor are default.

```
<?php
class MyClass {
  public $name;
  public $age;

  // Default constructor with default values
  public function __construct($name = "John Doe", $age = 30) {
    $this->name = $name;
    $this->age = $age;
  }

  public function displayInfo() {
    echo "Name: " . $this->name . ", Age: " . $this->age . "\n";
  }
}

// Instantiate with default values
$o1 = new MyClass();
$o1->displayInfo(); // Output: Name: John Doe, Age: 30

// Instantiate with provided values
$o2 = new MyClass("Alice Smith", 25);
$o2->displayInfo(); // Output: Name:
```

Output

Name: John Doe, Age: 30 Name: Alice Smith, Age: 25

Alice Smith, Age: 25 ?>	

Parameterized Constructor:

In parameterized constructor __construct() method takes one and more parameters. You can provide different values to the parameters.

Example:

<!DOCTYPE html>
<html>
<body>
 	
<?php
class Fruit {
  public $name;
  public $color;
 	
  function __construct($name) {
  $this->name = $name;
  }
  function get_name() {
  return $this->name;
  }
}
 	
$apple = new Fruit("Apple");
echo $apple->get_name();
?>
 	
</body>
</html>
	Output
	Apple

Destructor

PHP Destructor method is used to destroy objects or release their acquired memory. A

destructor is called automatically when the object is created. Usually, it is called at end of the script. The destructor method does not take any arguments, The destructor does not return any data type. This all process is handled by Garbage Collector.

Properties:

- **__destruct()** method does not take any parameter.
- **__destruct()** method will not have any return type.
- This method works exactly the opposite of the **__construct** method in PHP.
- **__destruct** gets called automatically at the end of the script.
- **__destruct()** method starts with two underscores (__).
- It is used to de-initialize existing objects.

Syntax:

```
function __destruct() {
   // Destroy objects or release memory.
}
```

Example:

```
<?php

class student {
  function __construct() {
     echo "This is a constructor<br>";
     echo "Object is initialized in
constructor<br>";
   }

   function __destruct() {
      echo "This is destruct<br>";
      echo "Object is destroyed in destructor";
   }
}

$subject = new student();

?>
```

Output:

This is a constructor
Object is initialized in constructor
This is destruct
Object is destroyed in destructor

PHP Access Modifiers or property visibility

In object-oriented programming, access specifiers are also known as access modifiers. These specifiers control how and where the properties or methods of a class can be accessed, either from inside the class, from a subclass, or from outside the class. PHP supports three primary access specifiers:

- public
- protected
- private

1. public

A public properties can be accessed from anywhere, from within the class, by inherited (child) classes, and from outside the class.. If a property is declared public, its value can be read or changed from anywhere in your script.

Example

<?php **class Demo {** **public $name = "ASSC";** **public function showName() {** **return $this->name;** **}** **}** **$obj = new Demo();** **echo $obj->name . "\n";** **echo $obj->showName();** **?>**	**Output** **ASSC** **ASSC**

2. private

A private property or method is accessible only within the class that declares it. It is not accessible in child classes or from outside the class.

Example:

<?php
class MyClass {
private $secret = "Top Secret";
 	
private function getSecret() {
return $this->secret;
}
 	
public function revealSecret() {
return $this->getSecret(); // Allowed internally
}
}
 	
$obj = new MyClass();
echo $obj->revealSecret(); // Works
echo $obj->secret; // Error: Cannot access private property
echo $obj->getSecret(); // Error: Cannot access private method
?>
	Output
	Top Secret
	Fatal error: Uncaught Error: Cannot access private property MyClass::$secret in /home/guest/sandbox/Solution.php:16
	Stack trace:
	#0 {main}
	thrown in /home/guest/sandbox/Solution.php on li...

3. protected

A protected property or method can only be accessed within the class itself and by inheriting classes (subclasses). It is not accessible from outside the class.

Example:

```php
<?php
class ParentClass {
   protected $message = "Hello from Parent";

   protected function showMessage() {
      return $this->message;
   }
}

class ChildClass extends ParentClass {
   public function getMessage() {
      return $this->showMessage();
   }
}

$obj = new ChildClass();
echo $obj->getMessage();
echo $obj->message;
?>
```

2.4.3 Inheritance and method overriding

PHP Inheritance

Inheritance in PHP is the ability of a class (known as a child class or subclass) to derive properties and methods from another class (known as a parent class or base class). Using inheritance, you can extend existing classes and modify or add new functionalities without changing the original code.

Syntax:

```php
class ParentClass {
   // Properties and methods
}
class ChildClass extends ParentClass {
   // Additional or overridden properties and methods
}
```

Types of Inheritance in PHP

1. Single Inheritance

PHP supports single inheritance, meaning a class can inherit from only one parent class at a time.

Example

```php
<?php
class A {
   public function sayHello() {
      echo "Hello from A\n";
   }
}
```

```
class B extends A {
   public function sayHi() {
      echo "Hi from B\n";
   }
}
$objA = new A();
$objA->sayHello();
$objB = new B();
$objB->sayHello();
$objB->sayHi();
?>
```

Output

```
Hello from A
Hello from A
Hi from B
```

2. Multilevel Inheritance

In multilevel inheritance, a class inherits from a class, which in turn inherits from another class.

Example

```
<?php
class A {
   public function greet() {
      echo "Hello from A"."<br>";
   }
}
class B extends A {
   public function welcome() {
      echo "Welcome from B"."<br>";
   }
}
class C extends B {
   public function message() {
      echo "Message from C"."<br>";
   }
}

$objA = new A();
$objA->greet();

$objB = new B();
$objB->greet();
$objB->welcome();

$objC = new C();
$objC->greet();
$objC->welcome();
```

Output

```
Hello from A
Hello from A
Welcome from B
Hello from A
Welcome from B
Message from C
```

$objC->message(); **?>**	

3. Multiple Inheritance

PHP does not support multiple inheritance directly through classes, but you can use traits to simulate multiple inheritance.

Alternatives to Multiple Inheritance:

1. **Traits:** Traits are a mechanism to reuse code in multiple independent classes. A trait is like a class that cannot be instantiated on its own, but its methods can be used by other classes.

Example:

Code:	Output:

<?php
trait Logger {
public function log($message) {
echo "Log: " . $message . "\n";
}
}
 	
trait Authenticator {
public function authenticate() {
echo "User authenticated\n";
}
}
 	
class User {
use Logger, Authenticator;
}
 	
$user = new User();
$user->log("User logged in");
$user->authenticate();
?>
	Log: User logged in
	User authsssenticated

Function Overriding in PHP

In function overriding, the parent and child classes have the same function name with and number of arguments.

Code:

```
<?php
  class Base {
    function display() {
      echo "Base class function declared final!";
    }
    function demo() {
      echo "Base class function!";
```

```
    }
  }

  class Derived extends Base {
    function demo() {
      echo "Derived class function!";
    }
  }

  $ob = new Base;
  $ob->demo();
  $ob->display();

  $ob2 = new Derived;
  $ob2->demo();
  $ob2->display();
?>
```

Output
Base class function!
Base class function declared final!
Derived class function!
Base class function declared final!

2.4.4 Exception Handling in PHP

An exception is an unexpected program result that can be handled by the program itself. Exception Handling in PHP is almost similar to exception handling in all programming languages.

PHP provides the following specialized keywords for this purpose.

- try: It represents a block of code in which exceptions can arise.
- catch: It represents a block of code that will be executed when a particular exception has been thrown.
- throw: It is used to throw an exception. It is also used to list the exceptions that a function throws, but doesn't handle itself.
- finally: It is used in place of a catch block or after a catch block basically it is put for cleanup activity in PHP code.

Syntax

```
try {
  code that can throw exceptions
} catch(Exception $e) {
  code that runs when an exception is caught
} finally {
  code that always runs regardless of whether an exception was caught
```

```
}
```

Exception handling in PHP:

- The following code explains the flow of normal try-catch block PHP:

Code

```
<!DOCTYPE html>
<html>
<body>

<?php
function divide($dividend, $divisor) {
  if($divisor == 0) {
    throw new Exception("Division by zero");
  }
  return $dividend / $divisor;
}

try {
  echo divide(5, 0);
} catch(Exception $e) {
  echo "Unable to divide. ";
} finally {
  echo "Process complete.";
}
?>

</body>
</html>
```

Output

```
Unable to divide. Process complete.
```

Creating a Custom Exception Class

To create a custom exception handler you must create a special class with functions that can be called when an exception occurs in PHP. The class must be an extension of the exception class. The custom exception class inherits the properties from PHP's exception class and you can add custom functions to it.

Code:

```
<?php
class customException extends Exception {
  public function errorMessage() {
    //error message
    $errorMsg = 'Error on line '.$this->getLine().' in '.$this->getFile()
    .': <b>'.$this->getMessage().'</b> is not a valid E-Mail address';
    return $errorMsg;
  }
}
```

```
$email = "jas.mca24@gmail.com";

try {
  //check if
  if(filter_var($email, FILTER_VALIDATE_EMAIL) === FALSE) {
    //throw exception if email is not valid
    throw new customException($email);
  }
}

catch (customException $e) {
  //display custom message
  echo $e->errorMessage();
}
?>
```

2.4.5 Input validation using regular expressions

What is a Regular Expression?

A regular expression is a sequence of characters that forms a search pattern. When you search for data in a text, you can use this search pattern to describe what you are searching for.

A regular expression can be a single character, or a more complicated pattern.

Regular expressions can be used to perform all types of text search and text replace operations.

Syntax

In PHP, regular expressions are strings composed of delimiters, a pattern and optional modifiers.

```
$exp = "/w3schools/i";
```

In the example above, / is the **delimiter**, *w3schools* is the **pattern** that is being searched for, and i is a **modifier** that makes the search case-insensitive.

The following table shows some regular expressions and the corresponding string which matches the regular expression pattern.

Regular Expression	**Matches**
Geeks	The string *"Geeks"*
^geeks	The string which starts with *"geeks"*
geeks$	The string which have *"geeks"* at the end.
^geeks$	The string where *"geeks"* is alone on a string.
[abc]	a, b, or c

[a-z]	Any lowercase letter
[^A-Z]	Any letter which is NOT a uppercase letter
(gif|png)	Either *"gif"* or *"png"*
[a-z]+	One or more lowercase letters
^[a-zA-Z0-9]{1, }$	Any word with at least one number or one letter
([ax])([by])	ab, ay, xb, xy
[^A-Za-z0-9]	Any symbol other than a letter or other than number
([A-Z]{3}|[0-9]{5})	Matches three letters or five numbers

Note: Complex search patterns can be created by applying some basic regular expression rules. Even many arithmetic operators like +, ^, – are used by regular expressions for creating little complex patterns.

Operators in Regular Expression:

Operator	Description
^	It denotes the start of string.
$	It denotes the end of string.
.	It denotes almost any single character.
()	It denotes a group of expressions.
[]	It finds a range of characters for example [xyz] means x, y or z .
[^]	It finds the items which are not in range for example [^abc] means NOT a, b or c.
– (dash)	It finds for character range within the given item range for example [a-z] means a through z.
| (pipe)	It is the logical OR for example x | y means x OR y.
?	It denotes zero or one of preceding character or item range.
*	It denotes zero or more of preceding character or item range.
+	It denotes one or more of preceding character or item range.
{n}	It denotes exactly n times of preceding character or item range for example n{2}.
{n, }	It denotes atleast n times of preceding character or item range for example n{2, }.
{n, m}	It denotes atleast n but not more than m times for example n{2, 4} means 2 to 4 of n.
\	It denotes the escape character.

Special Character Classes in Regular Expressions:

Special Character	Meaning
\n	It denotes a new line.
\r	It denotes a carriage return.
\t	It denotes a tab.
\v	It denotes a vertical tab.
\f	It denotes a form feed.
\xxx	It denotes octal character xxx.
\xhh	It denotes hex character hh.

Shorthand Character Sets: Let us look into some shorthand character sets available.

Shorthand	Meaning
\s	Matches space characters like space, newline or tab.
\d	Matches any digit from 0 to 9.
\w	Matches word characters including all lower and upper case letters, digits and underscore.

- **preg_match () function**

This function searches string for pattern, returns true if pattern exists, otherwise returns false. Usually search starts from beginning of subject string. The optional parameter offset is used to specify the position from where to start the search.

Syntax:

```
int preg_match( $pattern, $input, $matches, $flags, $offset )
```

Parameters:

Parameter	Description
pattern	The regular expression pattern (in quotes and slashes, like "/abc/")
subject	The input string to search
matches *(optional)*	Array to store the matches
flags *(optional)*	Modifiers like PREG_OFFSET_CAPTURE
offset *(optional)*	Where to start the search in the string

Returns:

- 1 if match is found
- 0 if no match
- FALSE on error

Example : Validating String with alphabets

```
$name = $_POST ["Name"];
if (!preg_match ("/^[a-zA-z]*$/", $name) ) {
  $ErrMsg = "Only alphabets and whitespace are
       allowed."; echo $ErrMsg;
} else {
  echo $name;
}
```

Form1.php

```
<?php
$ErrMsg = "";
$name = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
```

```
    $name = $_POST["Name"];

    // Validate name: only letters and whitespace allowed
    if (!preg_match("/^[a-zA-Z\s]*$/", $name)) {
        $ErrMsg = "Only alphabets and whitespace are allowed.";
    } else {
        echo "<h3>Valid Name: $name</h3>";
    }
}
?>

<h2>Enter Your Name</h2>
<form method="post" action="">
    Name: <input type="text" name="Name" value="<?php echo htmlspecialchars($name);
?>">
    <span style="color:red;"><?php echo $ErrMsg; ?></span><br><br>
    <input type="submit" value="Submit">
</form>

</body>
</html>
```

Example : Validating numeric

```
$mobileno = $_POST ["Mobile_no"];
if (!preg_match ("/^[0-9]*$/", $mobileno) ){
    $ErrMsg = "Only numeric value is allowed.";
    echo $ErrMsg;
} else {
    echo $mobileno;
}
```

Form1.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Mobile Number Validation</title>
</head>
```

```
<body>

<?php
$ErrMsg = "";
$mobileno = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $mobileno = $_POST["Mobile_no"];

  // Validate: Only numeric values (0-9)
  if (!preg_match("/^[0-9]*$/", $mobileno)) {
    $ErrMsg = "Only numeric value is allowed.";
  } else {
    echo "<h3>Valid Mobile Number: $mobileno</h3>";
  }
}
?>

<h2>Enter Your Mobile Number</h2>
<form method="post" action="">
  Mobile No: <input type="text" name="Mobile_no" value="<?php echo htmlspecialchars($mobileno); ?>">
  <span style="color:red;"><?php echo $ErrMsg; ?></span><br><br>
  <input type="submit" value="Submit">
</form>

</body>
</html>
```

Example: Validating Email

```
$email = $_POST ["Email"];
$pattern  =  "^[_a-z0-9-]+(\.[_a-z0-9-]+)*@[a-z0-9-]+(\.[a-z0-9-]+)*(\.[a-z]{2,3})$^";
if (!preg_match ($pattern, $email) ){
  $ErrMsg = "Email is not valid.";
      echo $ErrMsg;
} else {
  echo "Your valid email address is: " .$email;
}
```

Form.php

```
<!DOCTYPE html>
<html>
<head>
  <title>Email Validation Form</title>
</head>
<body>

<?php
$ErrMsg = "";
$email = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $email = $_POST["Email"];

  // Corrected regex pattern for email
  $pattern = "/^[_a-z0-9-]+(\.[_a-z0-9-]+)*@[a-z0-9-]+(\.[a-z0-9-]+)*(\.[a-z]{2,3})$/i";

  if (!preg_match($pattern, $email)) {
    $ErrMsg = "Email is not valid.";
  } else {
    echo "<h3>Your valid email address is: $email</h3>";
  }
}
?>

<h2>Enter Your Email</h2>
<form method="post" action="">
        Email: <input type="text" name="Email" value="<?php echo htmlspecialchars($email); ?>">
  <span style="color:red;"><?php echo $ErrMsg; ?></span><br><br>
  <input type="submit" value="Submit">
</form>

</body>
</html>
```

Example: Phone Number (Indian format, 10 digits)

```
$phone = "9876543210";
if (preg_match("/^[6-9]\d{9}$/", $phone)) {
   echo "Valid phone number";
} else {
   echo "Invalid phone number";
}
```

Username (Alphanumeric, 3–16 characters)

```
$username = "user123";
if (preg_match("/^[a-zA-Z0-9_]{3,16}$/", $username)) {
   echo "Valid username";
} else {
   echo "Invalid username";
}
```

Password (Min 8 characters, at least 1 letter and 1 number)

```
$password = "pass1234";
if (preg_match("/^(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d]{8,}$/", $password)) {
   echo "Valid password";
} else {
   echo "Invalid password";
}
```